Smart File Assistant

In Python

Study Course

B198c16 Software & AI

By

Chehab Hany Mohamed Elsayed Elsayed Elmenoufi

Student Number: GH1034223

Under the Guidance of

Prof. Hiba Khaled

GitHub URL: https://github.com/Chippo90/Software-and-AI

Video Recording URL: https://youtu.be/FtScd7dd1yI

Gisma University of Applied Sciences

Berlin, Germany

June 2026

Table of Contents

Abstract

The Smart File Assistant is an application developed to make file management easier and simpler. It automates the analysis of folders and directories by tracking changes over time and using a rule-based AI to provide recommendations. The system identifies large or old files and suggests archiving them. The Graphical User Interface (GUI) allows users to easily search, sort, and act on these suggestions, making it a practical tool for maintaining an organized workspace.

1. Introduction

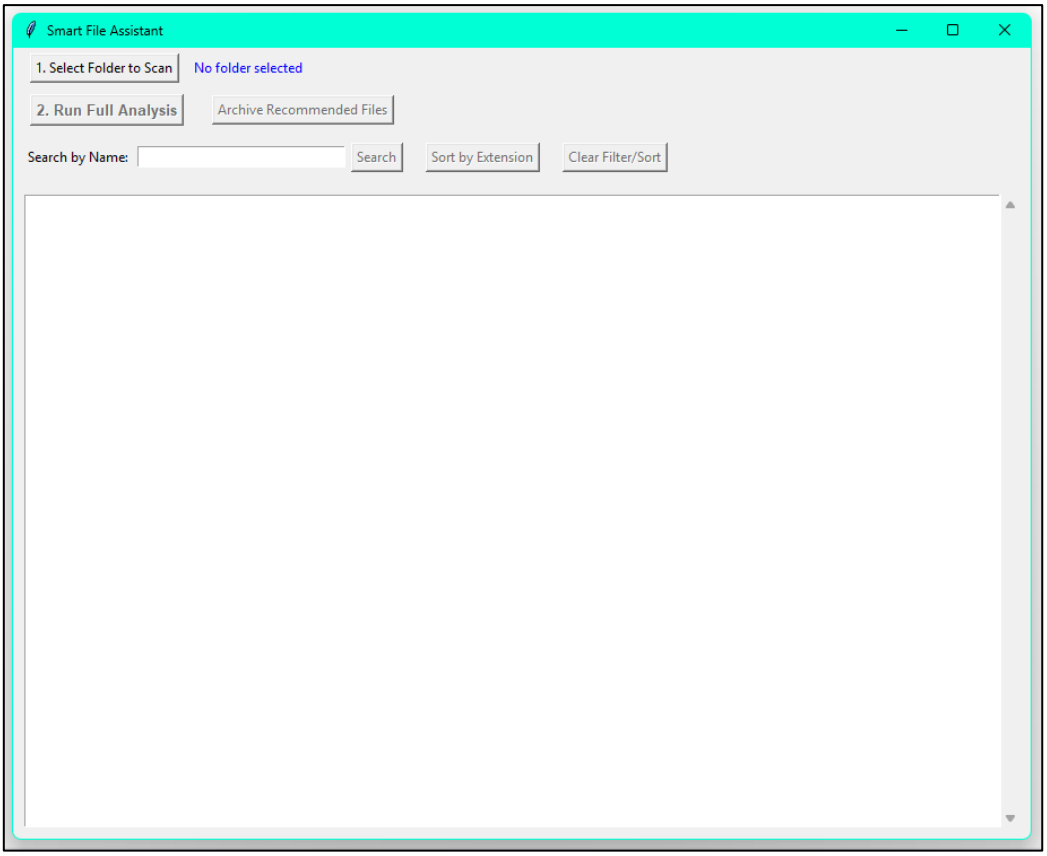
In a world filled with digital information, the number of personal and work files that accumulate on computers is huge; leading to disorganization, long search time, and inefficient storage. Manually organizing these files is a task that is postponed most of the time by users. The Smart File Assistant was developed as a solution to this problem, designed to bring automation and intelligence to file management.

The project was designed to implement the core logic of the provided plan, which is an AI system able to analyze file data to identify patterns and inefficiencies. The system intelligence can apply a set of rules to flag files that are ready for archive, like very large media files or old documents.

This report provides a review of the project cycle. It shows the system architecture from concepts and components, provides the implementation of each function, and offers the challenges faced and the strategies applied to overcome them.

2. System Architecture

The architecture of the Smart File Assistant follows an event driven model centered around GUI and separating the presentation logic from the backend layer.



2.1 Concept

The system is divided into four layers, making sure that each part of the application has a specific responsibility.

2.1.1 Presentation Layer “GUI”

Responsible for all user interaction, it is built with **“Tkinter”** and contains all visual elements. The role is to capture button clicks and display data from the layers below in a readable format.(*tkinter — Python interface to Tcl/Tk*, no date)

2.1.2 Control Layer “Event Handling”

This layer acts between the GUI and the backend. The App class is the primary controller, having methods that are triggered by user events. **“Run Analysis”** is the main function in this layer that manages the workflow.

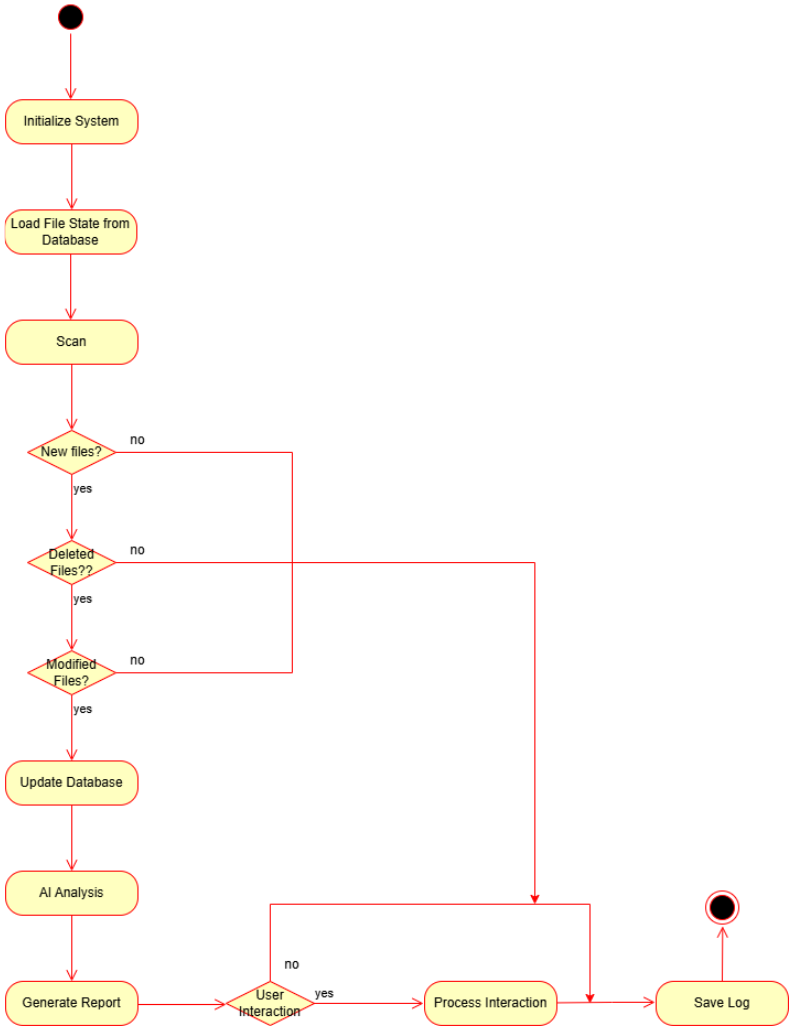
2.1.3 Backend Layer “Core Engine”

This layer is completely decoupled from the GUI and contains all the logic for file processing. It includes modules for scanning the file system, managing the state log, applying AI rules, and performing file operations.

2.1.4 Data Layer

This includes the user's file system, which is read during a scan, and the persistent **“smart_assistant_state.json”** file, which serves as the application memory.(*Python JSON*, no date; *Create a JSON Representation of a Folder Structure*, 18:55:48+00:00)

2.2 Activity Diagram



3. Implementation

The application was built starting with the core logic and step by step adding layers of in the GUI.

3.1 Core Engine

The foundation of the application is a set of functions that handle all data processing.

3.1.1 Analyze Folder

This function is the primary scanner. It uses **"os.walk"** to transfer every subdirectory and file within the given folder path. For each file, it calls **"os.stat"** to retrieve its data. It gives a dictionary, where each key is a file path and the value is another dictionary containing its size and last modified timestamp. This structure was chosen for the average time complexity for lookups.(*Python os.walk()*, no date; *Python os.stat() Method*, no date)

```
def analyze_folder(folder_path):  1 usage
    """Scans a folder and returns its current state."""
    current_state = {}
    for root, _, files in os.walk(folder_path):
        for file_name in files:
            if file_name == STATE_FILE_NAME:
                continue
            file_path = os.path.join(root, file_name)
            try:
                stats = os.stat(file_path)
                current_state[file_path] = {
                    'size': stats.st_size,
                    'last_modified': stats.st_mtime
                }
            except FileNotFoundError:
                continue
    return current_state
```

3.1.2 Load State & Save State

These functions handle persistence. **"Save State"** uses the **"JSON"** library to serialize the dictionary produced by **"Analyze Folder"** into text file. **"Load state"** does the reverse. Error handling was built in to manage cases where the file might be missing or corrupted, ensuring the app can always start when needed.(*os.path — Common pathname manipulations*, no date)

```
#Core Logic Functions
def load_state(folder_path):  1 usage
    """Loads the last saved state for a specific folder from the JSON log file."""
    state_file_path = os.path.join(folder_path, STATE_FILE_NAME)
    if not os.path.exists(state_file_path):
        return {}
    try:
        with open(state_file_path, 'r') as f:
            return json.load(f)
    except (json.JSONDecodeError, IOError):
        return {}


def save_state(folder_path, state_data):  1 usage
    """Saves the current state of the folder to the JSON log file."""
    state_file_path = os.path.join(folder_path, STATE_FILE_NAME)
    try:
        with open(state_file_path, 'w') as f:
            json.dump(state_data, f, indent=4)
    except IOError as e:
        print(f"Error: Could not save state file. Reason: {e}")
```

3.1.3 Detect Changes

This function lies at the center of the tracking. Instead of looping and comparing lists, it converts the file paths from the old and new states into set objects. This allows for efficient set operations:(*Python Sets*, 13:19:31+00:00)

- **Added Files:** (new files – old files)
- **Deleted Files:** (old files – new files)
- **Modified Files:** A check is performed at the intersection of both sets to see if the last modified timestamp has changed.

```
def detect_changes(old_state : {keys, __getitem__} , new_state : {keys, __getitem__} ):  1 usage
    """Compares two states to find added, deleted, and modified files."""
    old_files, new_files = set(old_state.keys()), set(new_state.keys())
    added = list(new_files - old_files)
    deleted = list(old_files - new_files)
    modified = [p for p in new_files.intersection(old_files) if
                old_state[p]['last_modified'] != new_state[p]['last_modified']]
    return added, deleted, modified
```

3.2 Artificial Intelligence Logic

The AI in this project is a rule based expert system.(*Rule-Based System in AI*, 18:36:08+00:00)

3.2.1 Get Files to Archive

This function embodies the AI logic. It iterates through the current state and applies two simple but powerful rules:

- **Size Rule:** Is the file size greater than 50MB?
- **Age Rule:** Is the number of days between now and the file's last modified date greater than 365 Days?

```
#Configuration
#The size in MB to consider a file "large".
LARGE_FILE_MB = 50
#The age in days to consider a file "old".
OLD_FILE_DAYS = 365
#The name of the log file.
STATE_FILE_NAME = "smart_assistant_state.json"
```

If a file meets either criterion, its path is added to a set to avoid duplicates and returned as a list of files recommended for archiving.

```
def get_files_to_archive(new_state : {items} ):  1 usage
    """This is the AI function. It identifies the *paths* of files that meet the archive criteria.This is used by the "Archive" button."""
    #Create an empty set to store the paths of files to archive, using a set avoids duplicates.
    files_to_move = set()
    #Get the current time for comparison.
    now = datetime.datetime.now()
    #Loop through all the files in the current state.
    for file_path, file_data in new_state.items():
        #Check if the file is large.
        if (file_data['size'] / (1024 * 1024)) > LARGE_FILE_MB:
            #If it is, add its path to our set.
            files_to_move.add(file_path)
        #Check if the file is old.
        last_modified_date = datetime.datetime.fromtimestamp(file_data['last_modified'])
        if (now - last_modified_date).days > OLD_FILE_DAYS:
            #If it is, add its path to our set.
            files_to_move.add(file_path)
    #Return the files to be moved as a list.
    return list(files_to_move)
```

3.3 Graphical User Interface

The GUI was built using the **"App class"** structure to encapsulate all widgets and their logic.(Shamim, 2025)

3.3.1 Layout

The layout was structured using **"Tkinter"** frame widgets to group controls. This makes the UI easier to manage and modify. The ".pack" was used for its simplicity.(*Python | pack() method in Tkinter*, 00:56:16+00:00)

```
#GUI Application Class
class App:  1 usage
    def __init__(self, root):
        #Store the main window.
        self.root = root
        #Set the window title.
        self.root.title("Smart File Assistant")
        #Set a larger default size for the new features.
        self.root.geometry("900x700")
        #Variable to store the path of the selected folder.
        self.folder_path = ""
        #Variable to hold the results of the last full scan.
        self.full_state = {}

        #Frame for Folder Selection
        top_frame = tk.Frame(root, padx=10, pady=5)
        top_frame.pack(fill='x')
        self.select_button = tk.Button(top_frame, text="1. Select Folder to Scan", command=self.select_folder)
        self.select_button.pack(side='left', padx=5)
        self.folder_label = tk.Label(top_frame, text="No folder selected", fg="blue")
        self.folder_label.pack(side='left', padx=5)
```

3.3.2 Widgets

Standard **"Tkinter"** widgets were used:

- "**Button**" for actions(*Python Tkinter*, 16:11:31+00:00)

```
#Main Action Frame
action_frame = tk.Frame(root, padx=10, pady=5)
action_frame.pack(fill='x')
self.run_button = tk.Button(action_frame, text="2. Run Full Analysis", command=self.run_analysis,
                            state='disabled', font=('Helvetica', 10, 'bold'))
self.run_button.pack(side='left', padx=5)
```

- "**Label**" for informative text(*Python Tkinter*, 16:11:31+00:00)

```
#Frame for Folder Selection
top_frame = tk.Frame(root, padx=10, pady=5)
top_frame.pack(fill='x')
self.select_button = tk.Button(top_frame, text="1. Select Folder to Scan", command=self.select_folder)
self.select_button.pack(side='left', padx=5)
self.folder_label = tk.Label(top_frame, text="No folder selected", fg="blue")
self.folder_label.pack(side='left', padx=5)
```

- "**Entry**" for the search bar(*Python Tkinter*, 16:11:31+00:00)

```
#Interactive for search and sort.
query_frame = tk.Frame(root, padx=10, pady=10)
query_frame.pack(fill='x')
#Create a label for the search bar.
tk.Label(query_frame, text="Search by Name:").pack(side='left')
#Create a text box for typing.
self.search_entry = Entry(query_frame, width=30)
self.search_entry.pack(side='left', padx=5)
#Create the "Search" button. It starts disabled.
self.search_button = tk.Button(query_frame, text="Search", command=self.search_files, state='disabled')
self.search_button.pack(side='left')
#Create the "Sort" button. It starts disabled.
self.sort_button = tk.Button(query_frame, text="Sort by Extension", command=self.sort_files, state='disabled')
self.sort_button.pack(side='left', padx=20)
#Create a "Clear" button to reset the view. It starts disabled.
self.clear_button = tk.Button(query_frame, text="Clear Filter/Sort", command=self.clear_filter,
                              state='disabled')
self.clear_button.pack(side='left')
```

- "**Scrolled Text**" for the main report area, which provides a built-in scroll for long reports.

```
#Report Area
#Create a text area to display all output.
self.report_area = scrolledtext.ScrolledText(root, wrap=tk.WORD, font=("Courier New", 10))
self.report_area.pack(pady=10, padx=10, fill="both", expand=True)
```

3.3.3 Event Handling

The command option of each button was bound to a method within the App class.

3.3.3.1 Select Folder

```
def select_folder(self):  1 usage
    """Called when the 'Select Folder' button is clicked."""
    path = filedialog.askdirectory()
    if path:
        self.folder_path = path
        self.folder_label.config(text=f"Selected: {path}")
        #Enable the main run button.
        self.run_button.config(state='normal')
        #Disable other buttons until a scan is complete.
        self.archive_button.config(state='disabled')
        self.search_button.config(state='disabled')
        self.sort_button.config(state='disabled')
        self.clear_button.config(state='disabled')
```

3.3.3.2 Run Analysis

```
def run_analysis(self):  2+ usages
    """Orchestrates the entire scan and report process."""
    if not self.folder_path: return

    self.report_area.delete( index1: 1.0, tk.END)
    self.report_area.insert(tk.END,  chars: f"Scanning {self.folder_path}...")
    self.root.update_idletasks()

    #Execute the core logic.
    old_state = load_state(self.folder_path)
    new_state = analyze_folder(self.folder_path)
    self.full_state = new_state  # Save the full results for interactive queries.
    changes = detect_changes(old_state, new_state)

    #AI part: get recommendations based on the scan.
    recommendations = self.get_recommendations_text(new_state)

    #Generate and display the report.
    report_text = generate_report_text(changes, new_state, recommendations)
    self.display_report(report_text)

    #Save the new state for the next time.
    save_state(self.folder_path, new_state)

    #Enable all buttons after we have data.
    self.archive_button.config(state='normal')
    self.search_button.config(state='normal')
    self.sort_button.config(state='normal')
    self.clear_button.config(state='normal')
```

3.3.3.3 Search Files

```
def search_files(self):  1 usage
    """Filters the displayed file list based on the search term."""
    #Get the search term from the entry box.
    search_term = self.search_entry.get().lower()
    #Do nothing if the search box is empty.
    if not search_term: return
    #Create a new dictionary containing only files that match the search term.
    filtered_state = {path: data for path, data in self.full_state.items() if
                      search_term in os.path.basename(path).lower()}
    #Generate a new report for the filtered files.
    report_text = generate_report_text( changes: ([], [], []), filtered_state,  recommendations: [])  # No changes or recs in this view.
    self.display_report(f"--- Search Results for '{search_term}' ---\n\n" + report_text)
```

3.3.3.4 *Sort Files*

```
def sort_files(self):  1 usage
    """Sorts the displayed file list by file extension."""
    #'sorted' can take a 'key' function. Here, we use a lambda to tell it to sort by the extension.
    #os.path.splitext('/path/to/file.txt') returns ('/path/to/file', '.txt'). We sort by the second part.
    sorted_state = dict(sorted(self.full_state.items(), key=lambda item : item.__getitem__() : os.path.splitext(item[0])[1]))
    report_text = generate_report_text( changes: ([], [], []), sorted_state,  recommendations: [])
    self.display_report("--- Files Sorted by Extension ---\n\n" + report_text)
```

3.3.3.5 *Clear Filters*

```
def clear_filter(self):  1 usage
    """Resets the view to the last full analysis report."""
    #Just re-run the report generation with the saved full state.
    recommendations = self.get_recommendations_text(self.full_state)
    report_text = generate_report_text( changes: ([], [], []), self.full_state, recommendations)
    self.display_report(report_text)
    self.search_entry.delete( first: 0, tk.END)  # Clear the search box.
```

3.3.3.6 *Archive File*

```
def archive_files(self):  1 usage
    """Moves all recommended files to a user-selected archive folder."""
    #First, identify which files need to be moved using our AI helper function [2].
    files_to_move = get_files_to_archive(self.full_state)
    if not files_to_move:
        messagebox.showinfo( title: "Archive Files",  message: "No old or large files to archive were found.")
        return

    #Ask user to confirm.
    if not messagebox.askyesno( title: "Confirm Archive",
                                message: f"Found {len(files_to_move)} files to archive. Do you want to proceed?"):
        return

    #Ask user to select an archive destination folder.
    archive_path = filedialog.askdirectory(title="Select a folder to move files into")
    if not archive_path: return  # Stop if the user cancels.

    #Move the files.
    moved_count = 0
    for file_path in files_to_move:
        try:
            # 'shutil.move' safely moves a file from a source to a destination.
            shutil.move(file_path, archive_path)
            moved_count += 1
        except Exception as e:
            print(f"Could not move {file_path}. Reason: {e}")

    #Show a final confirmation message.
    messagebox.showinfo( title: "Archive Complete",  message: f"Successfully moved {moved_count} out of {len(files_to_move)} files.")

    #Automatically run the analysis again to show the updated folder state.
    self.run_analysis()
```

3.4 Integration of Backend and GUI

The “Run Analysis” method serves as the main controller. It executes the analysis in order:

1. Displays a "Scanning..." message to the user.

```
self.report_area.delete( index1: 1.0, tk.END)
self.report_area.insert(tk.END,  chars: f"Scanning {self.folder_path}...")
self.root.update_idletasks()
```

2. Calls “Load State” to get the previous state.

```
old_state = load_state(self.folder_path)
```

3. Calls “Analyze Folder” to get the current state.

```
new_state = analyze_folder(self.folder_path)
```

4. Calls ‘Detect Changes” to compare the two states.

```
changes = detect_changes(old_state, new_state)
```

5. Calls recommendation logic to get a list of suggestions.

```
recommendations = self.get_recommendations_text(new_state)
```

6. Calls “Generate Report Text” to format all this data into a string.
```
report_text = generate_report_text(changes, new_state, recommendations)
```
7. Updates the widget with the final report.
```
self.display_report(report_text)
```
8. Calls “Save State” to persist the new state for the next run.
```
save_state(self.folder_path, new_state)
```

The features **“Search Files”** and **“Sort Files”** were implemented to operate on the scanned data stored, making sure they do not require another scan.
The “Archive Files” method uses **“shutil”** for file operations and **“Message Box”** for user confirmations.(*tkinter.messagebox — Tkinter message prompts*, no date; *shutil — High-level file operations*, no date)

3.5 Technologies Used

- **Programming Language**: Python 3
- **GUI Framework**: Tkinter(*GuiProgramming*, no date)
- **Core Libraries:** os, json, shutil, datetime

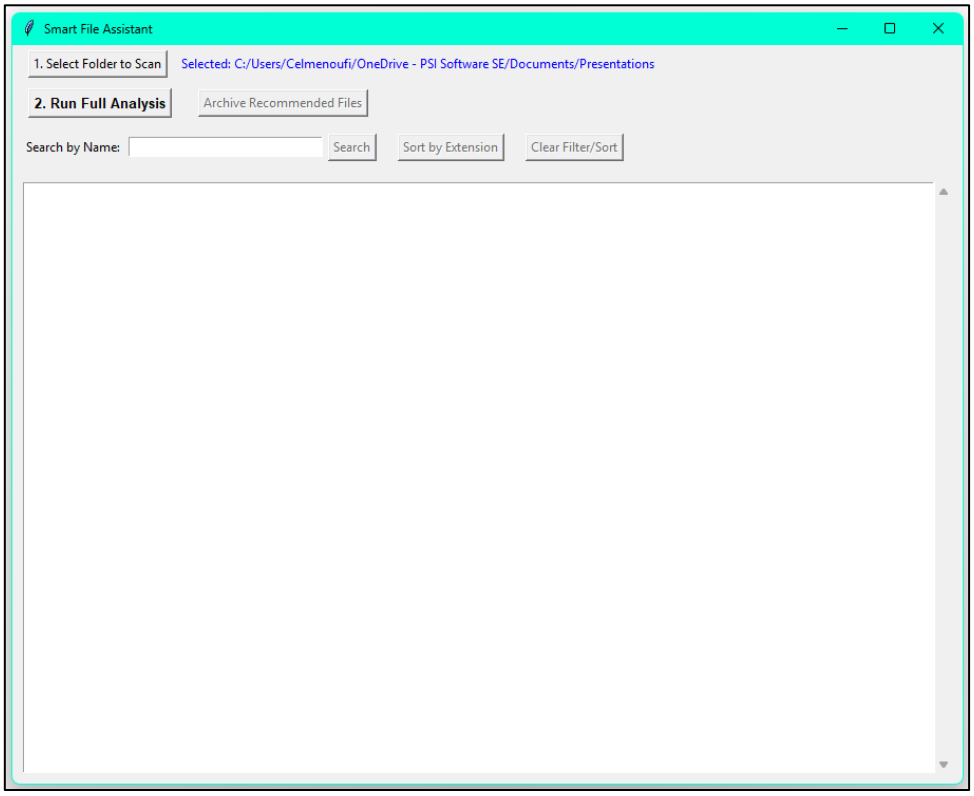
4. Results

4.1 Overview

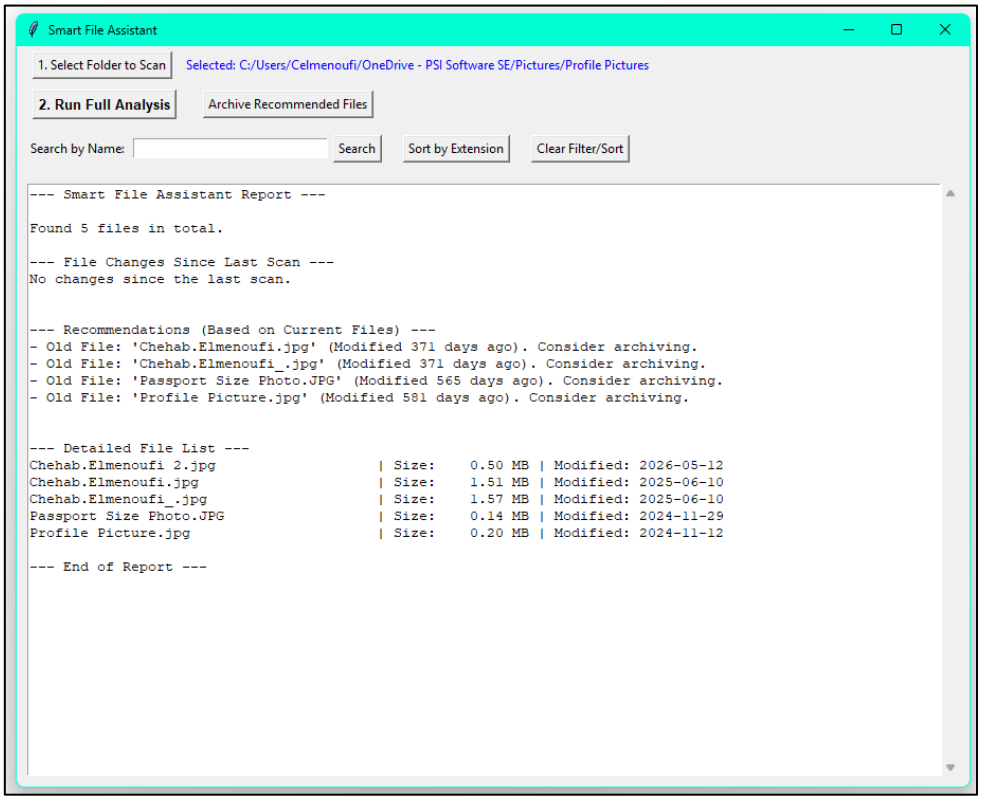
The project showed in a fully functional application that successfully meets all its objectives. It provides accurate change tracking, generates valuable AI-driven recommendations, and offers a set of interactive tools that empower the user to act on those insights efficiently and safely.

4.2 Screenshots

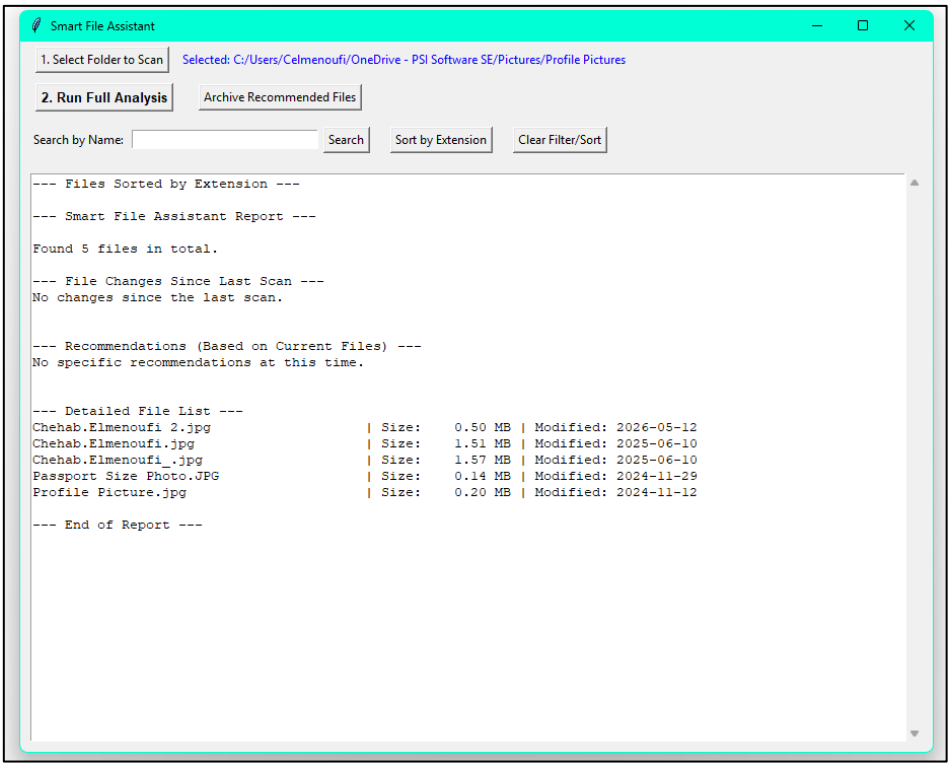
4.2.1 After Choosing a Folder



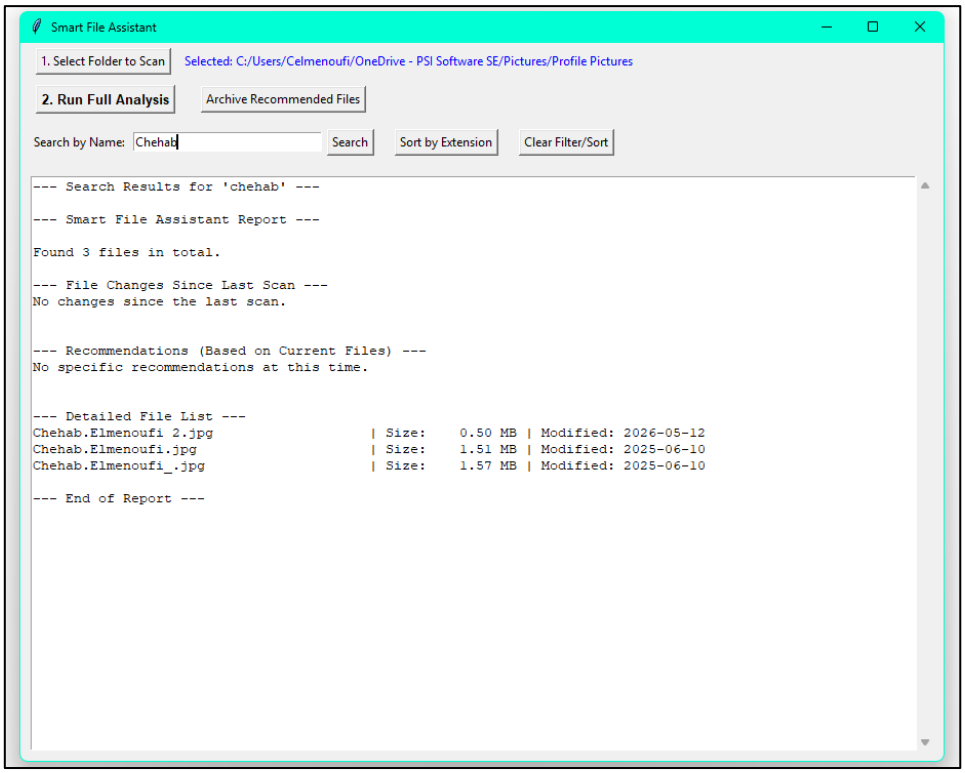
4.2.2 Folder Analysis Result



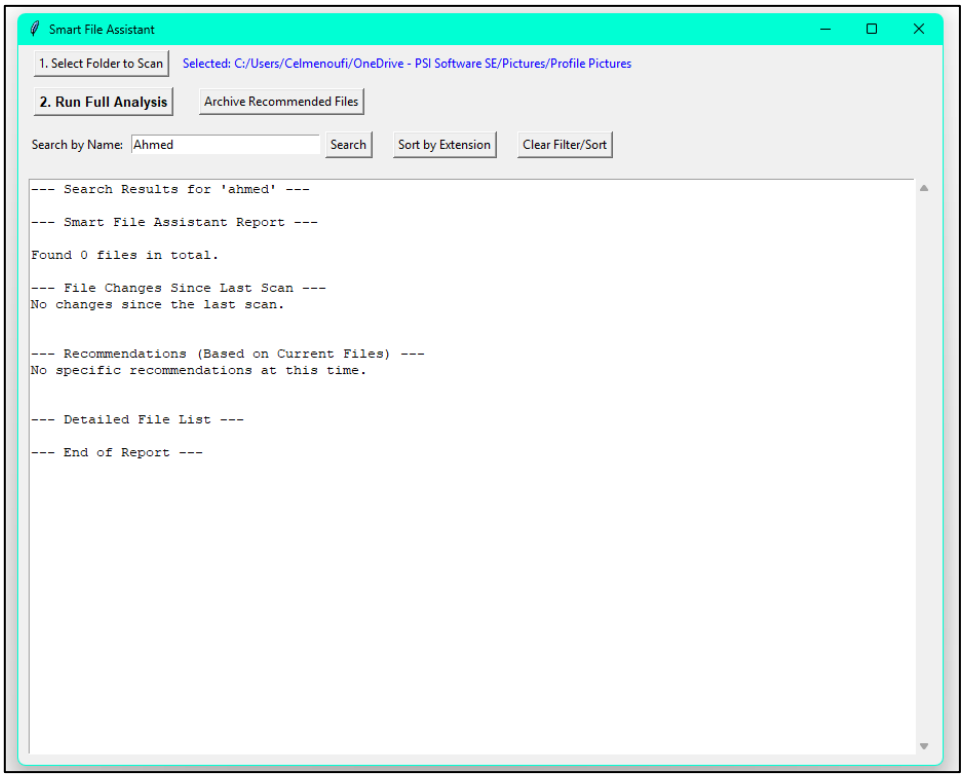
4.2.3 Applying Sorting by Extension



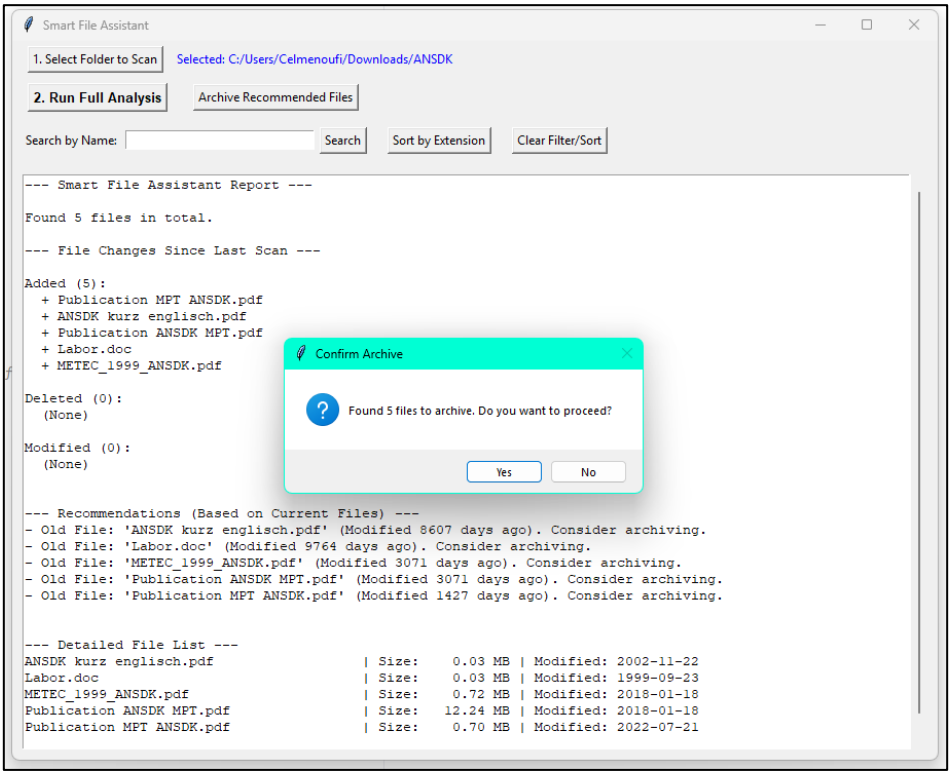
4.2.4 "Valid" Search Result



4.2.5 “Invalid” Search Result



4.2.6 Asking User Confirmation to Archive



5. Challenges and Solutions

5.1 Challenge #1: GUI Unresponsive During Scans

- **Description**: When scanning a directory with thousands of files, the I/O operations would block “Tkinter” main event loop, causing the application window to freeze.
- **Solution**: Before starting the scan in **“Run Analysis”,** the GUI is immediately updated with a "Scanning..." message, and **“Root Update Idle Tasks”** is called. This forces the UI to redraw before the blocking operation begins, assuring the user that the program is working fine.(*What’s the difference between “update” and “update_idletasks” in Tkinter?*, no date)

5.2 Challenge #2: Comparison of Directory States

- **Description**: To find added and deleted files would involve nested loops comparing two lists of file paths, which would be unacceptable slow for large directories.
- **Solution**: By converting the lists of file paths into sets, finding the differences becomes a fast operation. This was critical for the application's performance.

5.3 Challenge #3: Ensuring Data Safety and Trust

- **Description**: The purpose is to help organize files, but one of its key features involves moving them. A bug in this feature could lead to accidental data loss, which would be catastrophic for users.
- **Solution**: A multi-layered safety strategy was employed.
  - **Robust File Operations**: Instead of **“os.rename”**, the “**shutil.move”** function was used, as it is more powerful and handles moving files across different disks and file systems reliably.(*Python | shutil.move() method*, 00:54:06+00:00)
  - **Explicit User Confirmation:** Before any files are moved, a **“Message Box”** dialog is displayed, stating how many files will be moved and asking for user confirmation. The operation is cancelled if the user clicks "No."
  - **Post-Action Feedback:** After the operation, a final **“Message Box”** dialog informs the user of the result.(*tkinter.messagebox — Tkinter message prompts*, no date)

6. Conclusion and Future Work

The Smart File Assistant project is proof of the power of combining simple, rule-based intelligence with a well-structured, user-centric design. It successfully transforms the manual process of file management into an automated operation. The application is a robust case in Python application, from backend logic to frontend GUI design.

Future work could explore more features like duplicate file detection, machine learning based file classification and full drive scan and analysis.

---

7. References

Create a JSON Representation of a Folder Structure (18:55:48+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/python/create-a-json-representation-of-a-folder-structure/ (Accessed: June 06, 2026).

GuiProgramming (no date). Available at: https://wiki.python.org/moin/GuiProgramming (Accessed: May 09, 2026).

os.path — Common pathname manipulations (no date) *Python documentation*. Available at: https://docs.python.org/3/library/os.path.html (Accessed: May 09, 2026).

Python | pack() method in Tkinter (00:56:16+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/python/python-pack-method-in-tkinter/ (Accessed: June 03, 2026).

Python | shutil.move() method (00:54:06+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/python/python-shutil-move-method/ (Accessed: June 04, 2026).

Python JSON (no date). Available at: https://www.w3schools.com/python/python_json.asp (Accessed: June 06, 2026).

Python os.stat() Method (no date). Available at: https://www.tutorialspoint.com/python/os_stat.htm (Accessed: May 11, 2026).

Python os.walk() (no date). Available at: https://www.w3schools.com/python/ref_os_walk.asp (Accessed: May 11, 2026).

Python Sets (13:19:31+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/python/sets-in-python/ (Accessed: May 15, 2026).

Python Tkinter (16:11:31+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/python/python-gui-tkinter/ (Accessed: June 11, 2026).

Rule-Based System in AI (18:36:08+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/artificial-intelligence/rule-based-system-in-ai/ (Accessed: April 17, 2026).

Shamim, W. (2025) "Understanding Python Classes, Objects, __init__, self, Inheritance, Polymorphism, Encapsulation…," *Medium*, 27 June. Available at: https://medium.com/@Shamimw/understanding-python-classes-objects-init-self-inheritance-polymorphism-encapsulation-f916ed99388e (Accessed: April 28, 2026).

shutil — High-level file operations (no date) *Python documentation*. Available at: https://docs.python.org/3/library/shutil.html (Accessed: June 04, 2026).

tkinter — Python interface to Tcl/Tk (no date) *Python documentation*. Available at: https://docs.python.org/3/library/tkinter.html (Accessed: June 09, 2026).

tkinter.messagebox — Tkinter message prompts (no date) *Python documentation*. Available at: https://docs.python.org/3/library/tkinter.messagebox.html (Accessed: June 02, 2026).

What's the difference between "update" and "update_idletasks" in Tkinter? (no date). Available at: https://www.tutorialspoint.com/article/what-s-the-difference-between-update-and-update-idletasks-in-tkinter (Accessed: June 12, 2026).
